Bitácora de Decisiones de Diseño: Avance 2

Terminal de Juegos Retro Distribuida con Mando Embebido

Equipo del proyecto final, Grupo 6
Ingeniería en Computación, Facultad de Ingeniería, UNAM

I. Bitácora de Decisiones de Diseño

I-A. Clase 15: Almacenamiento de archivos

Pregunta: ¿Qué datos de su proyecto deben sobrevivir a un corte de energía? ¿En qué partición y sistema de archivos viven, y cuánto tiempo de datos pierde su sistema si la alimentación se interrumpe?

Respuesta aplicada al proyecto:
El mando casi no guarda nada: el firmware ocupa 41.7 KB de los 2 MB de Flash (2.0 %) y no tiene sistema de archivos. Toda la configuración son constantes que se fijan al compilar.

En la consola, lo delicado es la microSD. Las ROMs y el sistema casi solo se leen, pero la telemetría sí podría escribir mucho si la guardáramos. Con los números de la Clase 11:

- 32 B × 60 Hz = 1,920 B/s, que llena un bloque de 4,096 B de ext4 cada 2.1 s.
- En una hora son 6.9 MB, o sea 1,688 bloques.
- Con un `fsync` por mensaje serían 216,000 escrituras forzadas por hora. Con uno por minuto serían 60, que es 3,600 veces menos.

Decisión: la telemetría no va a la microSD. Vive en `/tmp`, que en nuestra Pi es `tmpfs` (RAM), y los puntajes se guardan solo al terminar la partida. El sistema está en `/dev/mmcblk0p2` con ext4. Como un arcade se desconecta de golpe, para la versión final queremos montar la raíz en solo lectura.

I-B. Clase 16: Administración de energía

Pregunta: ¿Cuánto tiempo debe funcionar su sistema sin alimentación externa? ¿Qué estados de energía puede usar y en qué momento haría un apagado controlado?

Respuesta aplicada al proyecto:
El arcade va conectado a la pared, así que la batería no es nuestro problema. Lo que importa es no perder cuadros y no dañar la microSD.

Medido en nuestra Pi 4 (Rev 1.5): frecuencia de 600 a 1,800 MHz, gobernador original `ondemand`, arranque de 21.7 s y `get_throttled` en `0x0` durante las pruebas.

Estimado, todavía sin medir: con 4 W de la Pi activa, unos 0.13 W del mando (5 V × 25 mA) y unos 0.5 W de la pantalla, serían 4.6 W. En una fuente de 5 V a 3 A (15 W) eso es el 31 %.

Decisiones:

- Entre partidas la consola se queda en inactivo, porque apagarla costaría 21.7 s en cada arranque.
- Dejamos `ondemand`. Si medimos cuadros perdidos, pasamos a `performance`.
- En la Pico, la adquisición digital de los 10 botones está totalmente delegada en hardware (PIO y un canal DMA dedicado) con un periodo de muestreo estricto de < 1 ms (1000 Hz). Core 1 se mantiene inactivo, delegando toda la gestión en Core 0 y periféricos autónomos.

I-C. Clase 17: Linux embebido

Pregunta: ¿Su proyecto necesita Linux?

Respuesta aplicada al proyecto:
La consola sí y el mando no.

- **Consola:** la Pi 4 tiene MMU y 1 GB de RAM. RetroArch necesita gráficos, audio, USB y sistema de archivos, y Linux ya trae esos controladores. Usamos Raspberry Pi OS Lite.
- **Mando:** el RP2040 es un Cortex-M0+ sin MMU y con 264 KB de RAM. Linux estándar no cabe ahí; la arquitectura se resuelve mediante firmware bare-metal asistido por hardware.

Al arrancar, un servicio de `systemd` lanzará RetroArch sin intervención. Ese servicio se hará en el Avance 3.

I-D. Clase 18: Elección del sistema operativo

Pregunta: ¿Qué sistema operativo usará su proyecto integrador: Linux, Windows IoT, Android, RTOS o ninguno? Indiquen el requisito que decide y un requisito que el sistema elegido no puede cumplir.

Respuesta aplicada al proyecto:
Ya con el módulo compilado revisamos la decisión de la Clase 7 con números propios, y se mantiene (Tabla I).

Cuadro I
Revisión de la elección de SO

Criterio	Mando: bare-metal	Consola: Linux
Determinismo	Requisito $<$ 1 ms (1000 Hz); transferencia DMA autónoma	Tiempo real blando: 16.6 ms por cuadro
Huella	41.7 KB de Flash, 5.6 KB estáticos de RAM	Reposo de 120 a 150 MB de 1 GB
Multitarea	Periféricos por hardware y Core 0	Procesos con planificador del kernel

I-E. Cambios de decisiones y su motivo

La Tabla II registra lo que cambió desde el Avance 1.

Cuadro II
DECISIONES QUE CAMBIARON

Antes	**Ahora**	**Motivo**
Bloque SPI del RP2040	Máquina PIO + 3 Canales DMA	El SPI no genera el pulso LD. La PIO a 1 MHz lee los 10 botones y 3 canales DMA asíncronos vuelcan los datos a SRAM/DPRAM sin que la CPU intervenga en la ruta crítica.
Uso de ambos núcleos	Core 0 activo, Core 1 inactivo	Toda la adquisición recae en PIO y DMA, dejando a Core 0 libre para telemetría y control USB, haciendo innecesario el uso de Core 1.
Flash < 40 KB	41.7 KB	La estimación no contaba la pila USB completa. Sigue siendo el $2\,\%$ de la Flash.
Antirrebote por software	Red RC por hardware	Filtrado físico con resistencias de $330\,\Omega$ y condensadores de 100 nF en paralelo.
8 botones	10 botones	Ajuste del panel de control definitivo para abarcar todo el de la arcade.
HDMI	TFT SPI de 3.5” (ST7796S)	Pantalla integrada al gabinete impreso.
CPU a 1.5 GHz	1.8 GHz	Nuestra Pi es Rev 1.5 y su máximo medido es 1,800 MHz.